Web App Pentest Checklist

This is the checklist I actually run on every web app engagement — built from what I've personally missed before, not copy-pasted from the OWASP testing guide. Every item has the reasoning attached, and every command is broken down flag by flag, because "run this" without "here's why" never stuck for me.

01 Recon & Mapping

Recon is the part I used to rush, and every time I rushed it I paid for it later — testing half an app while an old staging subdomain sat there untouched, still running last year's code with none of the new patches. So before I open Burp, I spend real time figuring out what actually exists first.

Passive recon

- [ ] **Confirm scope** in writing — hosts, IP ranges, subdomains, out-of-scope assets, rules of engagement, testing window. No scope confirmation, no testing. I don't care how tempting an out-of-scope host looks.
- [ ] Enumerate subdomains and DNS records before touching the app directly — old marketing pages and forgotten staging environments run weaker protections than the main app, and they're usually still in scope.
- [ ] Check certificate transparency logs and WHOIS/ASN data for related infrastructure — a cert log leaks every subdomain that's ever had a certificate issued for it, including internal tools someone assumed were hidden.
- [ ] Search for exposed source, secrets, or config in public repos and paste sites — a leaked API key or a committed `.env` file can skip you past half of this checklist in one shot.
- [ ] Fingerprint the stack: server headers, cookies, error pages, JS bundle names, CMS/framework tells — knowing what's actually running tells me which vulnerability classes are worth my time instead of guessing blind.

```
subfinder -d target.tld -silent | httpx -title -status-code -tech-detect
```

`subfinder -d target.tld`	pulls every subdomain it can find from passive sources — cert logs, DNS datasets, search indexes — without ever touching the target directly, so this step is invisible to them.
`-silent`	drops subfinder's own banner and log lines, so only clean subdomain names get piped forward to the next tool.
`-title`	grabs each live host's page title, so I can tell what an app probably is without opening every single one in a browser.
`-status-code`	shows the HTTP status per host, so I can sort live (200) from gated (401/403) from dead (404) at a glance.
`-tech-detect`	fingerprints the framework/CMS/server from response headers and page markers, telling me where to focus first.

Active mapping

- [ ] Crawl authenticated and unauthenticated app states separately — a crawler with no session cookie only ever sees the login page, so it misses almost the whole app.
- [ ] Walk every role (anon, user, admin, tenant-scoped) and diff the resulting sitemaps — the pages that show up in the admin diff but nowhere else are usually the least-tested ones.
- [ ] Pull JS bundles and grep for hidden routes, feature flags, and API paths — frontend code has to know about an endpoint to call it, so the bundle is basically a leaked map of the API.
- [ ] Brute-force common paths/backup files (`.git`, `.env`, `.bak`, admin panels) against scope, not blindly across the internet — a stray `.git` folder can hand you the entire source tree.
- [ ] Build a request inventory in your proxy (Burp/ZAP) grouped by feature, not just by URL — grouping by feature is how I make sure every workflow gets tested end to end, not just the URLs I happened to click.

02 Authentication

Auth is the front door. If there's one way to skip it, walk through it as someone else, or never really get logged out, everything downstream is compromised too — no amount of good code further in the app saves you from a broken lock on the way in. Most of the real bugs here live in the paths nobody designs on purpose: password reset, "remember me," the SSO callback.

- Username/email enumeration via login, registration, password reset, and error-message timing differences — a different error message (or even a slightly different response time) for "wrong password" versus "no such account" hands an attacker a validated target list for credential stuffing.
- Password policy: minimum strength enforced, no client-side-only validation, no silent truncation — a check that only lives in JavaScript is not a check; hitting the API directly skips it entirely.
- Brute-force / credential-stuffing protection: rate limiting, lockout, CAPTCHA — and that it applies per-account *and* per-source, because a limit that only counts per-IP does nothing against a spray from rotating IPs (I've watched this exact bypass play out in a real SOC investigation).
- Password reset token: single-use, short-lived, bound to the account, not leaked via Referer header or in a redirect URL — a token sitting in the Referer header leaks to any third-party resource the reset page happens to load.
- MFA: can it be bypassed by hitting the post-auth endpoint directly, replaying an old code, or downgrading to a non-MFA flow? MFA usually gets bolted onto the login form and then forgotten on the token-refresh or "trusted device" endpoint next to it.
- OAuth/SSO: `state` parameter validated, redirect_uri allow-listed exactly (not just prefix-matched), token audience checked — a prefix-matched redirect_uri (`https://target.com*`) is an open redirect waiting to hand your auth code to an attacker-controlled host.
- JWT: algorithm confusion (`alg: none`), weak/guessable signing secret, missing expiry validation, key confusion (RS256→HS256) — key confusion works because the RS256 public key is, by definition, public; if the server ever verifies a token using that same string as an HMAC secret, anyone can forge a token with it.
- "Remember me" and long-lived tokens don't outlive a password change or explicit logout — a password change that doesn't revoke existing long-lived tokens means the "fix" for a compromised account didn't actually fix anything.

03 Session Management

Once someone's authenticated, the session token is effectively their identity for the rest of the visit. I treat it with the same suspicion as a password, because in practice that's exactly what it is.

- Session identifiers are high-entropy, random, and never derived from predictable user data — anything guessable (timestamp, incrementing counter, hashed username) turns session hijacking into simple guesswork.
- Cookies: `Secure`, `HttpOnly`, and an appropriate `SameSite` value are all present — not just one of them, since each flag closes a different hole (plaintext transport, JS/XSS theft, cross-site request riding) and missing any one still leaves that hole open.
- Session fixation: pre-auth session ID is invalidated and replaced on successful login — otherwise an attacker who hands a victim a known session ID before login just walks into that same session once the victim authenticates.
- Logout actually invalidates the session server-side, not just the client-side cookie — a logout that only clears the browser cookie means the old token still works if it was ever captured.
- Concurrent session handling matches the app's stated policy (single-session vs. multi-device) — if the UI claims "log out everywhere," I make sure that button actually kills every session and not just the current one.
- Idle and absolute session timeouts are enforced server-side — a client-side timer is just a suggestion; the server has to be the one that actually expires the token.

04 Access Control & IDOR

> **Why this is the highest-yield section for me:** broken access control is consistently the most common high-severity finding I write up, and scanners barely catch any of it — every single item below has to be tested by hand, per role, per object. If I only had time for one section, this is the one I wouldn't skip.

- [ ] **Horizontal:** swap object IDs (order, invoice, profile, ticket) between two accounts of the *same* role — this is the single most common bug I find, and it's almost always a missing "does this object belong to this user?" check on the backend.
- [ ] **Vertical:** replay a low-privilege session against an admin/privileged endpoint or UI action — if the only thing hiding the admin action is that the button isn't rendered for regular users, it isn't hidden at all.
- [ ] Test every HTTP verb per endpoint, not just the one the UI uses — a `GET` may be checked while `PATCH` / `DELETE` on the same route isn't, because whoever wrote the auth check often only thought about the verb the frontend happens to send.
- [ ] Test IDs that are sequential, UUID, and hashed/encoded the same way — encoding is not authorization; a base64'd ID is just as guessable as the number underneath it once you decode it once.
- [ ] Multi-tenant apps: confirm tenant/org isolation on every list, search, and export endpoint, not just detail views — a search or export endpoint is an easy place for a developer to forget the tenant filter that every other endpoint has.
- [ ] Force-browse directly to feature-flagged or "not yet visible" UI states — a feature flag that only hides a menu item client-side is not an access control, it's a UI preference.
- [ ] Check that access control is enforced server-side even when the client hides the button/menu item — I assume every client-side restriction is decorative until I've proven the server enforces it too.

05 Input Validation & Injection

This is the category everyone thinks of first when they hear "pentest," and it's still worth doing properly even though it's the most automated part of the job — a scanner finds the obvious cases, but I've found real SQLi and SSTI in fields the scanner never reached because they were three steps into a workflow, not on the first page.

SQL / NoSQL injection CRIT

- Every parameter that reaches a query — including headers, cookies, and JSON body fields, not just the query string. A query built from a header value is exactly as injectable as one built from `?id=`.
- Error-based, boolean-blind, and time-blind confirmation when error output is suppressed — if the app swallows the SQL error, a consistent response delay from a `SLEEP()` payload is still proof, just slower to get.
- NoSQL: operator injection (`$ne`, `$gt`) in JSON bodies where a string was expected — sending an object where the backend expected a string password can turn `"password": "x"` into `"password": {"$ne": null}`, which most drivers happily evaluate as "not null," i.e. always true.

```
sqlmap -u "https://target/app?id=1" --batch --level=3 --risk=1 --dbs
```

`-u "...?id=1"`	the target URL, with the parameter I suspect is injectable already sitting in it.
`--batch`	take sqlmap's default answer to every prompt instead of stopping to ask — I want this to run unattended, not babysat.
`--level=3`	test more injection points and payload variations than the default (level 1), including cookies and some headers — the default misses a lot of what actually gets exploited in the wild.
`--risk=1`	stick to safe payloads — I deliberately don't raise this, since higher risk levels include heavier time-based and OR-based payloads that can lock accounts or hang the app.
`--dbs`	once injection is confirmed, list the actual database names — this is what turns "sqlmap said yes" into a screenshot I can put in a report.

I still confirm manually after this. sqlmap tells me something is there; it doesn't tell me what a client will believe, and a raw tool log is a worse piece of evidence than three clean manual requests.

Cross-site scripting

- Reflected: any request parameter echoed into the response without contextual encoding — "reflected" just means the payload never has to touch a database, it round-trips in a single request/response.
- Stored: content that persists and renders back to other users (comments, profile fields, filenames) — this is the more dangerous variant, since one submission can hit every visitor, not just the one who sent it.
- DOM-based: trace `location` / `postMessage` / `document.referrer` into sinks like `innerHTML`, `eval`, `document.write` — this class never touches the server at all, so it's invisible to any tool that only looks at HTTP traffic.
- Test inside every context separately: HTML body, attribute, JS string, URL, CSS — each needs different payload shaping, because a payload that escapes an HTML attribute usually does nothing inside a `<script>` block, and vice versa.

Server-side injection

- **SSTI:** template syntax (`{{7*7}}`, `${7*7}`) in any field that might be rendered through a template engine — if it comes back as `49` instead of literal text, the engine is executing what I typed, and that usually escalates straight to remote code execution.
- **Command injection:** anywhere the app shells out — file conversion, ping/traceroute utilities, image processing — any feature that "runs a system tool on your input" is a candidate, because that's exactly what a shell call looks like under the hood.

- **XXE:** XML parsers with external entities enabled; try file read and blind OOB exfil via DTD — this one's quietly common because a lot of XML libraries ship with external entity resolution turned on by default, and nobody remembers to turn it off.
- **Deserialization:** any endpoint accepting a serialized object (Java, PHP, Python pickle, .NET ViewState) — deserializing untrusted input means the attacker gets to define what object gets constructed, and a "helpfully" gadget-rich object graph turns into code execution.
- **Path traversal:** file-serving or upload/download endpoints — test encoded and double-encoded `../`, since a filter that strips one plain `../` often doesn't catch the URL-encoded or double-encoded version.

06 SSRF & Request Forgery

SSRF and CSRF get lumped together here because they're both "the server does something on my behalf that it shouldn't trust." One tricks the server into fetching somewhere it shouldn't; the other tricks it into acting on a request it should have rejected.

- Any feature that fetches a URL server-side: webhooks, "import from URL," link previews, PDF/screenshot generators, avatar-from-URL — anywhere the server is the one making the outbound request is a candidate, because the server usually has network access I don't.
- Reach cloud metadata endpoints (`169.254.169.254`) and internal-only hosts/ports from those features — on AWS/GCP/Azure that address serves the instance's temporary credentials to anything running on the box, so a working SSRF there can mean stealing the cloud account's own keys, not just reading a local file.
- Bypass naive filters: alternate IP encodings, redirects (301 to an internal host), DNS rebinding — a filter that just blocklists `127.0.0.1` and `localhost` misses decimal/octal IP forms and a URL that 301-redirects to an internal address after the check already passed.
- **CSRF:** state-changing requests without a per-session anti-CSRF token, or one that isn't actually validated server-side — a token that's generated but never checked on submit is worse than no token at all, because it gives false confidence in a report review.
- Confirm CORS: reflected `Access-Control-Allow-Origin` combined with `Allow-Credentials: true` is a finding, not a config note — together those two headers tell every other website on the internet it's allowed to read this app's authenticated responses on a victim's behalf.

07 Business Logic

> **No scanner finds these.** I have to actually use the app the way a real user (or a motivated abuser) would, then break the order, the quantity, or the assumption the workflow depends on. This is the slowest section and also the one I enjoy the most.

- Price/quantity manipulation: negative values, decimal rounding, currency mismatch, coupon stacking — a negative quantity can flip a charge into a refund if nobody ever imagined a customer would type a minus sign.
- Race conditions on anything limited or single-use: coupon redemption, inventory, "claim once" actions — fire requests concurrently, because a check-then-act pattern ("has this code been used? no? mark it used") only fails when two requests land inside that same gap at once.
- Workflow/state bypass: skip a required step by calling a later endpoint directly (e.g. confirm payment without completing checkout) — the UI enforcing step order is not the same as the API enforcing it.
- Rate limits and quotas actually apply per resource they claim to protect, not just at the API gateway edge — a gateway-level limit on requests-per-minute doesn't stop someone from exhausting a specific resource (like free trial credits) through a slower, patient script.
- File/export size and type limits enforced server-side, not just in the upload widget — the widget is a courtesy to the browser, not a security control.

08 File Upload

Upload features are popular because they're one of the few places a tester gets to hand the server a whole file instead of a single parameter — more surface, more chances something downstream trusts it too much.

- Extension check bypass: double extensions, null-byte tricks, case variation, alternate stream (`::$DATA`) — a check for `.php` at the end of the filename is easy to dodge with something like `shell.php.jpg` if the server picks the wrong extension to trust.
- Content-type/magic-byte checks vs. actual parsed file type — upload a polyglot, since a file can legitimately be valid as two formats at once (a GIF header followed by PHP code, for instance), passing whichever check looks at the front of the file.
- Where uploads are served from: same origin as the app (stored XSS risk) vs. a properly isolated storage origin — an uploaded HTML/SVG file served from the app's own origin executes with the app's own cookies available to it.
- Server-side processing of uploaded files (image resize, PDF parse, zip extract) for injected exploits (ImageTragick-style, zip-slip, XXE via SVG) — anything that "processes" what I upload is a program parsing untrusted input, which is exactly the situation that produces memorable CVEs.
- Overwrite existing files via a crafted filename/path in the upload request — a filename like `../../config/settings.php` tests whether the server actually sanitizes the name or just trusts what the client sends.

09 API Testing (REST / GraphQL)

APIs get less scrutiny than the pages built on top of them, mostly because nobody's clicking around in Postman the way they click around a website. That gap is exactly why I spend real time here.

- Every endpoint from the OpenAPI/Swagger spec is tested, not only the ones the frontend calls — a spec often documents endpoints an internal tool or a mobile app uses that the web frontend never touches.
- Mass assignment: send extra JSON fields (`role`, `isAdmin`, `balance`) and see if the backend binds them — this works when a framework auto-maps the whole request body onto a model, and the "only admins can set role" check happens somewhere that extra field skips right past.
- API versioning: an old, less-restricted version of an endpoint still reachable — `/v1/` often gets forgotten once `/v2/` ships, but forgotten doesn't mean removed.
- Rate limiting and pagination limits are enforced, and can't be bypassed by header/casing tricks — I've seen a rate limiter keyed on a client-supplied header, which is just asking to be reset by changing that header.
- **GraphQL:** introspection enabled in production; batched queries bypassing rate limits; excessive nesting/query-depth DoS; field-level authorization gaps that object-level checks miss — GraphQL's flexibility cuts both ways: the same feature that makes it powerful for developers makes it easy to ask for fields nobody intended to expose.

```
curl -s https://target/graphql -H 'Content-Type: application/json' \
  -d '{"query":"{__schema{types{name,fields{name}}}}"}' | jq .
```

`-s`	silent — suppress curl's progress meter so the terminal only shows the actual response, not a status bar cluttering the output.
`-H 'Content-Type: application/json'`	tells the server the body is JSON, not form data, so it actually parses the query instead of rejecting or mangling it.
`-d '{"query":"..."}'`	the actual GraphQL introspection query — it literally asks the schema to describe itself: every type, every field.
`| jq .`	pretty-prints the JSON response, since a raw introspection result is a wall of minified text that's unreadable without it.

If introspection is left on in production, this single request hands over the entire schema — including fields never exposed in the UI — so I'm not blind-guessing field names for the rest of the assessment.

10 Configuration & Transport

None of this is glamorous, and it's exactly the section a rushed test skips. It's also the section a client's own infra team can fix in an afternoon, so I never leave it out.

- TLS: no weak ciphers/protocols, valid cert chain, HSTS present with a real max-age — a short or missing HSTS max-age means a single plain-HTTP visit is all it takes to fall back to an interceptable connection.
- Security headers present and not just copy-pasted boilerplate: `CSP`, `X-Frame-Options` / `frame-ancestors`, `X-Content-Type-Options` — I check these actually apply to the pages that need them, not just the homepage someone tested once.
- Default credentials on any exposed admin/management interface — "nobody would leave this on default" is wrong often enough that it's always worth the thirty seconds to check.
- Directory listing, backup files, and source maps not exposed alongside production assets — a `.map` file next to a minified JS bundle hands you readable source code for free.
- Cloud storage buckets referenced by the app aren't publicly listable/writable — a listable bucket turns "guess one filename" into "just browse the whole thing."

- Verbose errors/stack traces disabled in the environment under test — a stack trace often names the exact framework version and internal file paths, doing an attacker's recon for them.

11 Client-Side & Storage

Everything the browser holds is everything an attacker gets if they ever land script execution on the page, so I look at what's actually being kept around client-side and whether it needed to be there at all.

- Sensitive data (tokens, PII) not persisted in `localStorage` where XSS would exfiltrate it directly — unlike a cookie, JavaScript can always read `localStorage`, so anything stored there is available to any script that ever runs on the page.
- `postMessage` handlers validate `origin` before trusting message content — a handler with no origin check accepts messages from literally any page that gets a reference to that window, not just the one it was written for.
- Third-party scripts and widgets scoped so a compromised one can't read the whole page's DOM/cookies — every third-party script is a dependency on that vendor's own security, whether anyone thought of it that way or not.
- Clickjacking: sensitive actions can't be framed and clicked through an overlay — if the page can be iframed onto an attacker's site, a transparent overlay can trick a user into clicking a real button they never saw.

12 Evidence & Reporting

A finding nobody can reproduce is just an opinion. I write every one of these up assuming the reader has never seen the app before, because on a big enough team, they haven't.

- Every finding has: request/response pair, reproduction steps, impact tied to the specific app (not a generic CVSS blurb), and a concrete fix — "this is XSS, CVSS 6.1" tells a developer nothing about what to actually change.
- Screenshots/PoCs redact other testers' and real users' data before they leave your machine — a proof-of-concept screenshot is not the place for someone else's real session token to show up.
- Chained findings (a low-severity info leak that enables a high-severity IDOR) are written up as a chain, not scattered as isolated low-severity items — reporting them separately hides the actual impact, which is the thing a client is paying to understand.
- Retest confirms the fix at the code/config level, not just that the original payload no longer fires — I've seen a "fix" that only blocked the one exact payload I sent, not the underlying bug, and it only shows up if you actually go looking the second time.

13 Quick Reference — Common Payload Starters

This is the table I actually pull up mid-test when I need something to paste in a hurry — a starting probe, not a finding on its own. Every one of these needs its impact confirmed before it goes in a report.

CLASS	STARTER PAYLOAD	WATCH FOR
SQLi	`' OR '1'='1`	Error text, row-count/behavior change
SQLi (blind)	`' AND SLEEP(5)--`	Consistent response delay
XSS	`"><svg onload=alert(1)>`	Unescaped in response body
SSTI	`{{7*7}} / ${7*7}`	Rendered as "49"
XXE	`<!ENTITY xxe SYSTEM "file:///etc/passwd">`	File contents in response
Path traversal	`..%2f..%2f..%2fetc%2fpasswd`	File contents, 500 vs 404 diff
Command inj.	`; id ; whoami`	Command output, timing delay
CRLF inj.	`%0d%0aSet-Cookie: x=1`	Injected header in response

Every payload here is a starting probe, not a finding. Confirm impact, avoid destructive follow-through, and stay inside the agreed scope and rules of engagement at all times.
